Week 1 - Preprocessing

Please run the cells of the notebook as you get to them while reading

In [89]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

1. Lesson on how to search for Python commands

Let's consider a few possible ways to learn about Python programming. Let's suppose you want to learn how to produce a short summary of the information in your DataFrame.

1. Your **instructor** could provide the information.

You could be provided with a lesson about functions like info() and describe(). If you have a pandas DataFrame called df, then you can summarize its contents using df.info() or df.describe(). df.info() provides a list of column names with their counts and data types. df.describe() will provide information such as the mean, min, max, standard deviation, and quantiles. Thus:

In [90]:
```
df = pd.DataFrame([[1, 4], [2, 5], [3, 6], [4, 7]], columns = ['A', 'B'])
df.describe()
```

Out[90]:

	A	B
count	4.000000	4.000000
mean	2.500000	5.500000
std	1.290994	1.290994
min	1.000000	4.000000
25%	1.750000	4.750000
50%	2.500000	5.500000
75%	3.250000	6.250000
max	4.000000	7.000000

In this describe() result, we see that the two columns A and B each have four elements. The means and other statistics are shown.

2. You could look up the information on **Google**.

If I Google the question "how do I briefly summarize the contents of a dataframe using Python," I receive the following link (among others), which discusses the describe() command mentioned above:

https://www.w3schools.com/python/pandas/ref_df_describe.asp

It also provide the complete usage information:

dataframe.describe(percentiles, include, exclude, datetime_is_numeric)

It explains that "percentiles" is set by default to [0.25, 0.5, 0.75] but we could change that. Let's try it! Since there are three intervals here rather than four, it might be more meaningful to ask about a 33rd and 67th percentile rather than 25, 50, and 75. We can use 1/3 for 0.33 and 2/3 for 0.67 to get the exact percentile values.

In [91]:
```
df = pd.DataFrame([[1, 4], [2, 5], [3, 6], [4, 7]], columns = ['A', 'B'])
df.describe(percentiles = [1/3, 2/3])
```

Out[91]:

	A	B
count	4.000000	4.000000
mean	2.500000	5.500000
std	1.290994	1.290994
min	1.000000	4.000000
33.3%	2.000000	5.000000
50%	2.500000	5.500000
66.7%	3.000000	6.000000
max	4.000000	7.000000

Apparently, the 50% value (the median) stays even though we did not specifically request it.

3. You could look up the official **documentation**.

Now that we know we want the pandas describe() function, try Googling: pandas documentation describe.

Here is the general documentation page for pandas:

https://pandas.pydata.org/docs/index.html

Here is the specific page for the describe() function:

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html

When I look at this, it appears to be showing the most recent (currently 2.2) version of pandas; this is shown in the upper right corner.

4. You could also ask **ChatGPT**.

Let's try it. ChatGPT, "how do I briefly summarize the contents of a dataframe using Python"

When I do this, ChatGPT mentions describe() among other options, but does not go into detail. However, I could ask it. ChatGPT, "tell me more about describe() in Python for summarizing dataframes."

Then, I get a good explanation of describe(), although it does not mention the percentiles option. One advantage of using Google or the documentation in addition of ChatGPT is that these sources may provide interesting information that does not directly answer our question. Thus, we might not have known about the various arguments, such as percentiles, if we only used ChatGPT. A second issue is that ChatGPT sometimes hallucinates (it makes up information). In general, by examining multiple sources - Google, documentation, and ChatGPT - we can get more information.

2. Weekly graph question

In Storytelling With Data, on page 1: examine the pie chart graph in the upper left corner of the graphs. Please write a short explanation of the pros and cons of this graph. What do you think of the choice of pie chart as a format? The color scheme? The legend? The title? How would you draw it differently if you were creating this graph?

The data could be represented better using a bar chart versus the pie chart. The pie chart was likely chosen because the data displayed is percentages adding up to 100%, which can be shown easily as the full pie chart. However, the pie chart makes it difficult to compare the quantities. Using a horizontal bar chart with a y axis showing the percentage or absolute count for each category would make it easier for comparison. Additionally, the title "Survey Results" is not effective since it misses context about what the survey was, and what is being measured in the chart. A better title would be "How do you feel about science?". The color scheme and legend are both done well. The colors contrast well and the legend labels make sense with an updated title. The one drawback of the labels is that they take up a lot of room in the chart. The follow-up charts in page 5 of Storytelling With Data have an improved legend which spans the top of the chart instead.

3. Homework - Bank Customers

I will begin by creating a file for you to analyze. I will show you all of the steps I used to create it. Please run this code in order to create and save a file about bank customers.

The numbered problems are for you to solve.

In [92]:
```
num_customers = 100
np.random.seed(0)
```

In [93]:
```
df_bank = pd.DataFrame(columns = ["CustomerID"])
```

In [94]:
```
df_bank["CustomerID"] = [str(x) for x in np.arange(num_customers)]
```

In [95]:
```
start = datetime(1950, 1, 1)
end = datetime(2024, 1, 1)
numdays = (end - start).days
random_days = np.random.randint(0, numdays, size = num_customers)
df_bank["BirthDate"] = start + pd.to_timedelta(random_days, unit='D')
df_bank["BirthDate"] = df_bank["BirthDate"].dt.strftime('%Y-%m-%d')
```

In [96]:
```
def make_ssn_string(num):
    ssn_str = f'{num:09}'
    return ssn_str[0:3] + "-" + ssn_str[3:5] + "-" + ssn_str[5:9]
ssn_vector_func = np.vectorize(make_ssn_string)
df_bank["SSN"] = ssn_vector_func(np.random.randint(0, 999999999, size = num_custome
```

In [97]:
```
df_bank["AccountID"] = np.random.randint(0, num_customers, size = num_customers)
```

In [98]:
```
random_days = np.random.randint(0, 365 * 80, size = num_customers)
df_bank["AccountOpened"] = (pd.to_datetime(df_bank["BirthDate"]) + pd.to_timedelta(
```

In [99]:
```
df_bank.loc[0, "BirthDate"] = "1980"
df_bank.loc[1, "BirthDate"] = "no date"
```

In [100…
```
df_bank.loc[2, "AccountID"] = np.nan
```

In [101…
```
df_bank["AccountType"] = np.random.choice(["checking", "savings", "cd"], size = num
```

Load the bank_customers.csv file. (There is no practical reason to save it, then load it - we're just demonstrating how this would be done.) I am calling the loaded df by a new name, df_bank_loaded, to make clear why it's not the same variable as the old df. Of course, in actuality the two contain the exact same data! But it's good to get in the habit of naming things carefully.

In [102…
```
df_bank.loc[num_customers - 1] = df.loc[0]
df_bank.to_csv("bank_customers.csv", index=False)
```

In [103…
```
df_bank_loaded = pd.read_csv("bank_customers.csv")
```

1. Use describe() and info() to analyze the data. Also, look at the first few rows.

In [104…
```
df_bank_loaded.describe()
```

Out[104…

	CustomerID	AccountID
count	99.000000	98.000000
mean	49.000000	46.551020
std	28.722813	27.679358
min	0.000000	0.000000
25%	24.500000	25.500000
50%	49.000000	42.000000
75%	73.500000	71.000000
max	98.000000	97.000000

In [105…

```
df_bank_loaded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   CustomerID     99 non-null     float64
 1   BirthDate      99 non-null     object 
 2   SSN            99 non-null     object 
 3   AccountID      98 non-null     float64
 4   AccountOpened  99 non-null     object 
 5   AccountType    99 non-null     object 
dtypes: float64(2), object(4)
memory usage: 4.8+ KB
```

In [106…

```
df_bank_loaded
```

Out[106…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
...	...	...	...	...	...	...
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings
99	NaN	NaN	NaN	NaN	NaN	NaN

100 rows × 6 columns

Suggested Google Search or ChatGPT prompt: "how do I use the describe function in python"

Example Google result: https://www.w3schools.com/python/pandas/ref_df_describe.asp

In [107…

```
# The first few rows
df_bank_loaded.iloc[0:5]
```

Out[107…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd

If you used describe() and info(), you now know that BirthDate and AccountOpened are strings. But we want them to be dates. Let's convert them to dates (or Timestamps in pandas). When we try this, we get a ValueError.

In [108…

```
try:
    df_bank_loaded["BirthDate"] = pd.to_datetime(df_bank_loaded["BirthDate"], forma
    print("It worked!")
except ValueError as e:
    print(f"ValueError for BirthDate: {e}")
```

```
ValueError for BirthDate: time data "1980" doesn't match format "%Y-%m-%d", at posit
ion 0. You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily
in exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element ind
ividually. You might want to use `dayfirst` alongside this.
```

In [109…

```
try:
    df_bank_loaded["AccountOpened"] = pd.to_datetime(df_bank_loaded["AccountOpened"
    print("It worked!")
except ValueError as e:
    print(f"ValueError for AccountOpened: {e}")
```

```
It worked!
```

The simple way to fix this is to remove the rows that have bad dates for BirthDate. I Googled:

"How to remove rows from a dataframe that have poorly formatted dates using python"

https://stackoverflow.com/questions/21556744/pandas-remove-rows-whose-date-does-not-follow-specified-format

This recommends that I verify that the date is a string of length 10, because YYYY-MM-DD has that length:

df1[df1.BirthDate.str.len() !=10]

In [110…

```
len(df_bank_loaded[df_bank_loaded.BirthDate.str.len() == 10])
```

Out[110…

```
97
```

In [111…

```
df_bank_loaded[df_bank_loaded.BirthDate.str.len() != 10].iloc[0:5]
```

Out[111…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
99	NaN	NaN	NaN	NaN	NaT	NaN

Now we can make this permanent, creating a new DataFrame df_bank_datefix. I am making a copy in order to ensure that df_bank_datefix is a new DataFrame rather than being a slice of the old one.

In [112…

```
df_bank_datefix = df_bank_loaded[df_bank_loaded.BirthDate.str.len() == 10].copy()
```

Test again:

In [113…

```
try:
    df_bank_datefix["BirthDate"] = pd.to_datetime(df_bank_datefix["BirthDate"], for
    print("It worked!")
```

```
except ValueError as e:
    print(f"ValueError: {e}")
```

```
It worked!
```

2. To check that it worked, use a summary function that will tell you if the BirthDate field is now a datetime type

In [114…

```
df_bank_datefix.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 97 entries, 2 to 98
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     97 non-null     float64
 1   BirthDate      97 non-null     datetime64[ns]
 2   SSN            97 non-null     object
 3   AccountID      96 non-null     float64
 4   AccountOpened  97 non-null     datetime64[ns]
 5   AccountType    97 non-null     object
dtypes: datetime64[ns](2), float64(2), object(2)
memory usage: 5.3+ KB
```

3. Check whether there are any null values in the DataFrame. If so, remove those rows or (if you prefer) fill in the value with an appropriate number.

First try at a Google search or ChatGPT prompt: "how do I find out if there are any null values in a pandas DataFrame?"

This page gives an answer. Unfortunately, it took my request too literally: it tells me only if there are any, and not which rows have them. On reflection, that's not really what I want - I think I asked the wrong question. I want to see the rows, not just *whether* there are any.

https://stackoverflow.com/questions/29530232/how-to-check-if-any-value-is-nan-in-a-pandas-dataframe

ChatGPT likewise doesn't give the answer I want - because I asked the wrong question.

Next try at a Google search or ChatGPT prompt: "how do I check which rows have null values in a pandas DataFrame?"

This page gives an answer:

https://stackoverflow.com/questions/36226083/how-to-find-which-columns-contain-any-nan-value-in-pandas-dataframe

ChatGPT also gives a good answer. I recommend looking at both of them!

Now try it on your own:

Suggested Google search or ChatGPT prompt: "how do I remove rows with null values in a pandas DataFrame?"

Suggested Google search or ChatGPT prompt: "how do I fill in null values in a pandas DataFrame?"

In [115…

```
df_bank_datefix.isna().sum()
```

Out[115…

```
CustomerID       0
BirthDate        0
SSN              0
AccountID        1
AccountOpened    0
AccountType      0
dtype: int64
```

In [116…

```
df_bank_datefix[df_bank_datefix.isna().any(axis=1)]
```

Out[116…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking

In [117…

```
df_bank_datefix_non_null = df_bank_datefix.dropna()
print(f'Length before dropping nulls: {len(df_bank_datefix)}')
print(f'Length after dropping nulls: {len(df_bank_datefix_non_null)}')
df_bank_datefix_non_null
```

```
Length before dropping nulls: 97
Length after dropping nulls: 96
```

Out[117…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

4. Find out if there are any duplicate rows (two rows exactly the same). List their row numbers. Then remove the duplicates

In [118…
```
df_bank_datefix_non_null[df_bank_datefix_non_null.duplicated(keep=False)]
```

Out[118…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

There are no strictly duplicate rows, but we can confirm by checking if any duplicate SSNs

In [119…
```
df_bank[df_bank.duplicated(subset="SSN", keep=False)]
```

Out[119…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

Suggested Google search or ChatGPT prompt: "how can I find out if there are any duplicate rows in a DataFrame using Python"

Again, Google provides me with a page that addresses the question:

https://saturncloud.io/blog/how-to-find-all-duplicate-rows-in-a-pandas-dataframe/

To remove the duplicates, do this search: "how can I remove the duplicate rows in a DataFrame using Python"

This leads me to the following documentation.

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

In [120…
```
# There were no duplicates so this does not do anything
df_bank_datefix_non_null = df_bank_datefix_non_null.drop_duplicates()
df_bank_datefix_non_null
```

Out[120…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

5. Check whether the customers all have unique AccountIDs. If not, provide the first example of a non-unique AccountId.

In [121…

```
df_bank_datefix_non_null[df_bank_datefix_non_null.duplicated(subset="AccountID", ke
```

Out[121…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
41	41.0	1996-08-30	560-38-9694	3.0	2073-11-13	savings
74	74.0	2023-01-19	086-36-1272	3.0	2093-12-24	cd

Suggested Google search or ChatGPT prompt: "how can I find the first non-unique item from a pandas Series in python"

By the way: why didn't I ask the question "how can I check whether the customers all have unique AccountIDs"?

The problem would be that Google and ChatGPT don't know what "customers" you are talking about. It's important to understand that the AccountIDs are a column of a DataFrame, and as such they are a Series. Therefore, we should use the correct vocabulary and ask about a Series. If you mess up and ask about a "list" instead of a Series, you *might* get an answer that still works. But it's better to get the vocabularly right.

It's important to add "in python" because this task could be performed in many languages.

ChatGPT gave me this suggestion: data[data.isin(data[data.duplicated()])].iloc[0] However, ChatGPT did not explain how this code worked and even claimed (falsely) that it was going to use the value_counts() function in the solution. So although the code is correct, I personally found ChatGPT's answer very confusing. You could, perhaps, ask ChatGPT to explain further how this code works.

ChatGPT, "How does this code work: data[data.isin(data[data.duplicated()])].iloc[0]"

On the other hand, Google leads me to the documentation for the duplicated() function:

https://pandas.pydata.org/docs/reference/api/pandas.Series.duplicated.html

Here, I can see that when I really need is data.duplicated(keep = False), where "data" should be the Series in question. However, this just gives me a Series of boolean values indicating which ones are duplicates. I have to somehow know that extracting the numerical values instead of a Series of booleans involves boolean indexing: data[data.duplicated(keep = False)].

So as usual, I'd suggest that a combination of Google, documentation, and ChatGPT will give you the best information.

6. Count how many distinct AccountIDs there are.

In [122…

```
len(df_bank_datefix_non_null['AccountID'].unique())
```

Out[122…

```
63
```

Suggested Google search or ChatGPT prompt: "how can I find out how many distinct items there are in a pandas Series using python"

This time Google provides me with a page that's specifically made to answer this question:

https://www.geeksforgeeks.org/how-to-count-distinct-values-of-a-pandas-dataframe-column/

7. Remove the duplicate AccountIDs so that each AccountID appears only once.

This will involve using data.duplicated() but this time without keep = False. We don't want to drop all duplicates; we want to leave one example of each value.

In [123…

```
df_bank_datefix_non_null_unique_accts = df_bank_datefix_non_null.drop_duplicates(su
df_bank_datefix_non_null_unique_accts
```

Out[123…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
88	88.0	1962-02-07	327-09-3267	97.0	1989-10-03	cd
90	90.0	1969-06-18	395-31-7064	34.0	2010-12-28	checking
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd

63 rows × 6 columns

In [124…

```
# Confirming no dupes left
df_bank_datefix_non_null_unique_accts[df_bank_datefix_non_null_unique_accts.duplica
```

Out[124…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

8. What are the mean, median, and mode customer age in years? (Rounding down to the next lower age.) Are there any outliers? (Customers with very large or very small ages, compared with the other ages?)

In [125…

```
df_bank_with_age = df_bank_datefix_non_null.copy()
df_bank_with_age
```

Out[125…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

In [126…

```
# https://stackoverflow.com/questions/26788854/pandas-get-the-age-from-a-date-examp
from datetime import date
def calculate_age(born):
    today = date.today()
    return today.year - born.year - ((today.month, today.day) < (born.month, born.d

df_bank_with_age['Age'] = df_bank_with_age['BirthDate'].apply(calculate_age)
```

In [127…

```
df_bank_with_age
```

Out[127…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	Age
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	22
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	40
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	17
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	50
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	9
...	...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings	56
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking	50
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd	36
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd	65
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings	50

96 rows × 7 columns

In [128…

```
df_bank_with_age['Age'].mean(), df_bank_with_age['Age'].median(), df_bank_with_age[
```

Out[128…

```
(np.float64(35.010416666666664),
 np.float64(34.0),
 0    29
 Name: Age, dtype: int64)
```

Suggested Google search or ChatGPT prompt: "how can I find out the mean, median, and mode of a pandas Series"

9. One-hot encode the AccountType column. This means creating a new "checking," "savings", and "cd" columns so that you can run machine learning algorithms.

In [129…

```
df1 = df_bank_datefix_non_null_unique_accts.copy()
df2 = df_bank_datefix_non_null_unique_accts.copy()

one_hot = pd.get_dummies(df1["AccountType"], dtype=int)
df2 = df2.join(one_hot)
df2.iloc[0:5]
```

Out[129…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	1	0
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	1	0
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	0	1
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	0	0
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	0	0

Now, change the cd, checking, and savings columns into integers.

I modified the original code to do this by using `dtype=int` argument

10. Are there any other data values that do not seem right? If not, give an example?

In [130…

```
df2.sort_values(by="BirthDate")
```

Out[130…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
39	39.0	1952-01-26	810-29-3625	67.0	1983-11-04	checking	0	1
40	40.0	1952-03-08	291-98-5506	82.0	1953-05-05	cd	1	0
33	33.0	1955-02-15	986-06-7499	93.0	1967-11-11	cd	1	0
28	28.0	1955-12-04	863-97-2245	40.0	1996-02-08	checking	0	1
35	35.0	1956-11-01	470-06-0415	95.0	2031-10-28	checking	0	1
...	...	...	...	...	...	...	...	...
37	37.0	2017-07-23	991-68-0187	88.0	2094-12-17	checking	0	1
54	54.0	2017-09-13	403-47-1401	37.0	2066-06-21	cd	1	0
59	59.0	2019-06-26	119-05-0326	89.0	2065-03-29	savings	0	0
62	62.0	2022-08-23	719-02-2091	51.0	2090-07-02	cd	1	0
61	61.0	2023-03-31	040-18-6728	53.0	2068-01-17	savings	0	0

63 rows × 9 columns

One issue with the data types is that AccountID is treated as a float, for example 4.0. We should be treating them as integers instead since they are always whole numbers

For the values themselves, we can see there are some issues with the AccountOpened feature. We see values in the future like "2093-12-24", which is impossible. Additionally, there are customers with recent BirthDates like "2023-12-26". It is unlikely that a customer would have an account this early in life unless their family opened it for them. We would want to evaluate the business rules to validate.

I don't think Google or ChatGPT alone will help you here. To answer the question, look at the columns and think about what relationships they should have with each other. For example, it seems reasonable to expect that BirthDate would be no earlier than 120 years ago (it's unlikely that a customer would be this old.) Now we can ask Google:

"How can I find out how long ago a pandas date is"

Google provides this helpful link, although it is not exactly the solution - you'll have to work with it a bit:

https://stackoverflow.com/questions/26072087/pandas-number-of-days-elapsed-since-a-certain-date

If you check, I think you'll find that all dates are more recent than 120 years ago. What about the AccountOpened columns? I see some obviously wrong dates there just by looking at the first few rows.

Along those same lines, are there any birth dates that are too recent? Do we think that any two year olds will have opened bank accounts? How common do you think this is in real life? How common is it in our data set? Can you detect the two year olds opening bank accounts using just one column, or do you need two columns?

11. Use Matplotlib and/or Seaborn to analyse the ages at which customers open their account. Is there a connection between the year they are born vs. the age at which they open the account? Graph this in whatever way you think is best.

I asked Google and ChatGPT: "How can I plot dates vs. dates in Matplotlib". This gave me a hard time at first - I had to tell ChatGPT it was giving me the wrong information because it tried to plot dates vs. numbers. Eventually, I found out that you plot dates vs. dates in the same way you'd plot numbers vs. numbers.

Think in terms of Storytelling With Data to plot these as best you can. Once you've seen the result, try to think of the best way to plot the data so as to show the user what you want them to see. Title the graph so as to display the lesson that you want the user to take away. Here are some options for the axes:

1. A scatter or line plot: On the x-axis, the date they are born. On the y-axis, the date they open the account.
2. A scatter or line plot: On the x-axis, the date they are born. On the y-axis, the age in years at which they open the account.
3. A scatter or line plot: On the x-axis, they year (integer) they are born. On the y-axis, the age in years at which they open the account.
4. A histogram: on the x-axis, the age at which they open the account.

Here is an example:

In [131…

```
df2
```

Out[131…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	1	0
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	1	0
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	0	1
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	0	0
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	0	0
...	...	...	...	...	...	...	...	...
88	88.0	1962-02-07	327-09-3267	97.0	1989-10-03	cd	1	0
90	90.0	1969-06-18	395-31-7064	34.0	2010-12-28	checking	0	1
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings	0	0
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking	0	1
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd	1	0

63 rows × 9 columns

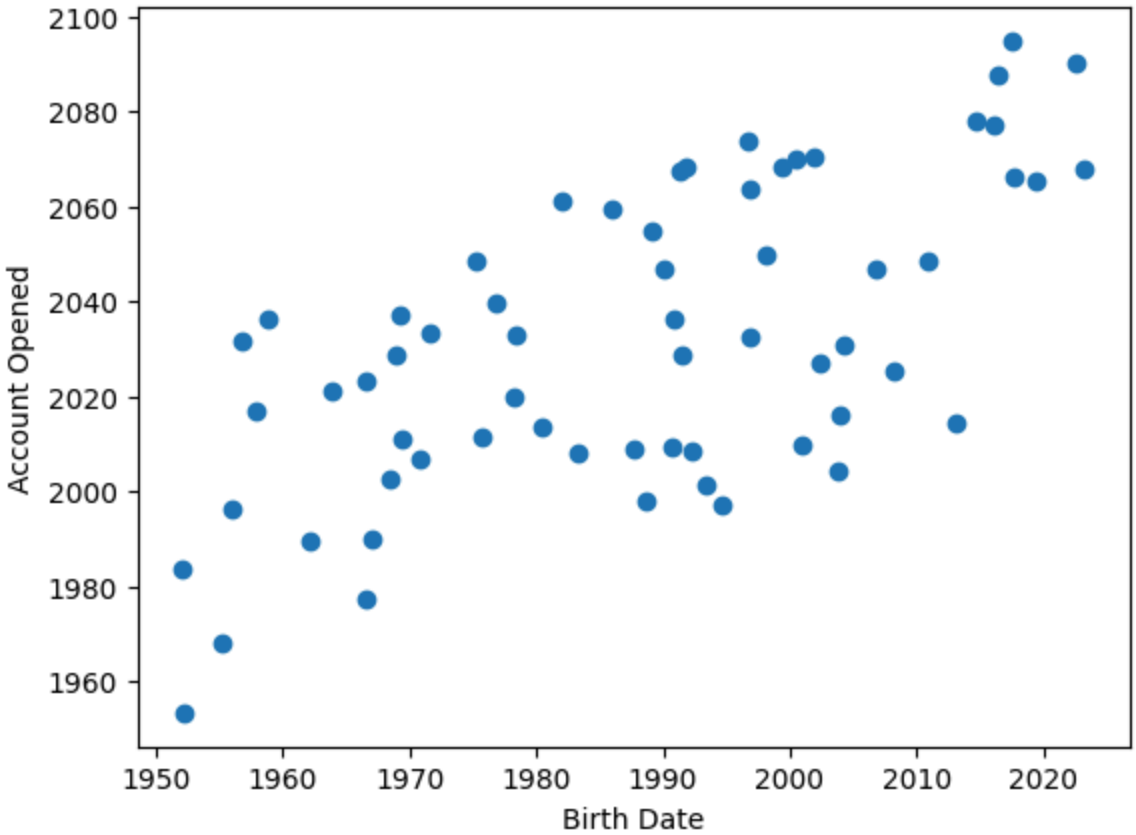
In [132…

```
import matplotlib.pyplot as plt

ax = plt.gca() # get an "Axes" object to draw on; gca stands for "get current Axes"
ax.scatter(df2["BirthDate"], df2["AccountOpened"]) # create a scatter plot based on
ax.set_ylabel("Account Opened") # label the y axis
ax.set_xlabel("Birth Date") # label the x axis
```

Out[132…

```
Text(0.5, 0, 'Birth Date')
```

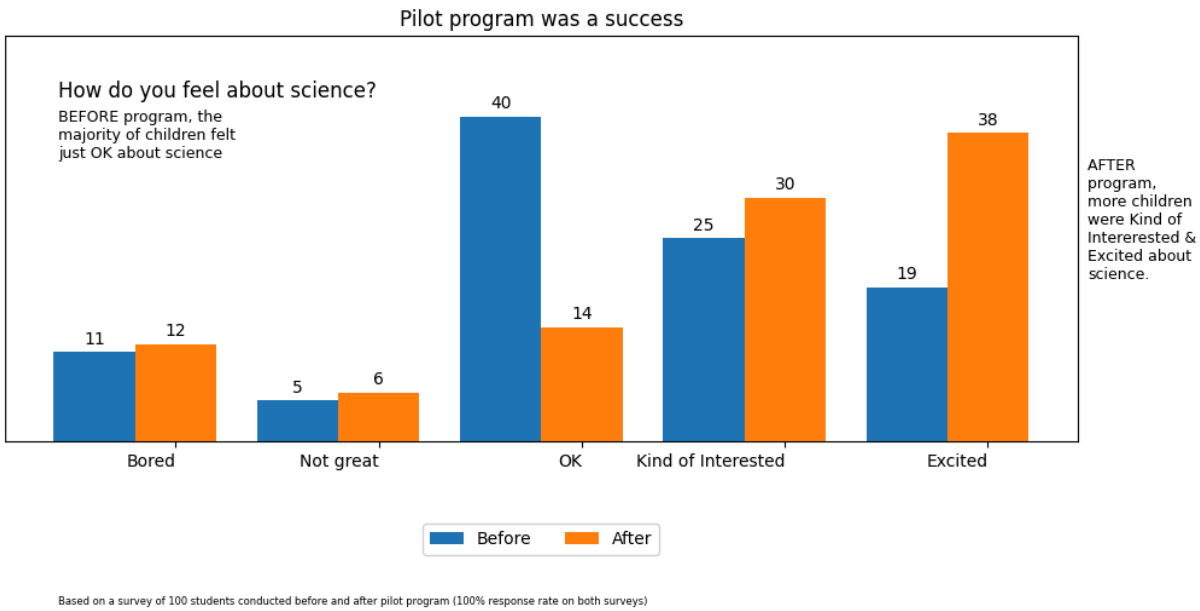


4. Storytelling With Data graph

Choose any graph in the Introduction of Storytelling With Data. Using matplotlib to reproduce it in a rough way. I don't expect you to spend an enormous amount of time on this; I understand that you likely will not have time to re-create every feature of the graph. However, if you're excited about learning to use matplotlib, this is a good way to do that. You don't have to duplicate the exact values on the graph; just the same rough shape will be enough. If you don't feel comfortable using matplotlib yet, do the best you can and write down what you tried or what Google searches you did to find the answers.

In [58]:
```
import matplotlib.pyplot as plt
import numpy as np

# Figure 0.5 (page 5)
import matplotlib.pyplot as plt
categories = ("Bored", "Not great", "OK", "Kind of Interested", "Excited")
pilot_program_data = {
    'Before': (11, 5, 40, 25, 19),
    'After': (12, 6, 14, 30, 38)
}

x = np.arange(len(categories))  # the label locations
width = 0.4  # the width of the bars
multiplier = 0
```

```
fig, ax = plt.subplots(layout='constrained', figsize=(10, 5))

for attribute, measurement in pilot_program_data.items():
    offset = width * multiplier
    rects = ax.bar(x + offset, measurement, width, label=attribute)
    ax.bar_label(rects, padding=3)
    multiplier += 1

ax.set_title('Pilot program was a success')
ax.set_xticks(x + width, categories, ha='right')
ax.legend(loc='lower center', ncols=2, bbox_to_anchor=(0.5, -0.3))
ax.set_ylim(0, 50)
ax.yaxis.set_visible(False)
ax.text(0.05, 0.85, "How do you feel about science?", horizontalalignment='left', f
ax.text(0.05, 0.7, "BEFORE program, the \nmajority of children felt \njust OK about
ax.text(1.01, 0.4, "AFTER \nprogram, \nmore children \nwere Kind of \nIntererested
ax.text(0.05, -0.4, "Based on a survey of 100 students conducted before and after p
None
```